

45 Questions Related to Python Programming

Beginner to Advanced | Questions 01 – 45
With Concept Explanations + Examples + Code

Section 1 : Python Basics

What is a Variable?

A variable is a named storage location in memory that holds a value. You can think of it like a labeled box — you put a value inside it and refer to it by its name. In Python, you do not need to declare the type of a variable. Python automatically understands whether it is a number, text, or anything else.

Example:

```
name = "Jarina" # string variable
age = 21 # integer variable
salary = 25000.50 # float variable
```

What is an Integer?

An Integer is a whole number — it has no decimal point. Examples: 1, 25, 100, -5, 0. In Python, integers can be positive or negative. They are used for counting, indexing, and calculations.

Example:

```
x = 10
y = -5
print(type(x)) #
```

What is a Float?

A Float is a number with a decimal point. Examples: 3.14, 25.5, -0.001. Floats are used when you need more precision, such as salary amounts, percentages, or scientific values.

Example:

```
pi = 3.14159
print(type(pi)) #
```

What is a String?

A String is a sequence of characters enclosed in single or double quotes. It is used to store text. Examples: 'Hello', 'Jarina', 'Python 3.12'. Strings support many built-in operations like slicing, joining, replacing, and finding.

Example:

```
name = "Jarina Choudhary"
print(name[0]) # J (first character)
```

```
print(name[-1]) # y (last character)
print(name.upper()) # JARINA CHOUDHARY
```

What is a Boolean?

A Boolean has only two possible values: True or False. It is used to represent conditions — something is either true or false. Booleans are the result of comparisons and are heavily used in if-else conditions.

Example:

```
is_active = True
is_admin = False
print(10 > 5) # True
print(10 == 5) # False
```

What is input()?

The input() function is used to take data from the user while the program is running. Whatever the user types is received as a string. If you need a number, you must convert it using int() or float().

Example:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

Questions — Section 1 : Basics

1. Print 'Hello, World!' in Python.

```
print("Hello, World!")
```

2. Declare variables of all data types and print them with their types.

```
name = "Jarina"
age = 21
salary = 25000.50
active = True
print(type(name), type(age), type(salary),
type(active))
```

3. Take name and age as input from user and print a greeting.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Hello {name}! You are {age} years old.")
```

4. Perform all arithmetic operations on two numbers.

```
a, b = 15, 4
print(a + b) # 19 Addition
print(a - b) # 11 Subtraction
print(a * b) # 60 Multiplication
print(a / b) # 3.75 Division
print(a // b) # 3 Floor Division
print(a % b) # 3 Modulus
print(a ** b) # 50625 Power
```

5. Take length and breadth as input and calculate area of rectangle.

```
l = float(input("Length: "))
b = float(input("Breadth: "))
area = l * b
print(f"Area of rectangle = {area}")
```

Section 2 : Conditions and Loops

What are Conditions (if / elif / else)?

Conditions allow your program to make decisions. The if statement checks whether something is True. If it is, that block of code runs. elif means 'else if' — it checks another condition if the first one was False. else runs when none of the above conditions are True. This is how programs choose different paths based on input or data.

Example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

What are Loops (for / while)?

A loop repeats a block of code multiple times. A for loop runs for a fixed number of times — for example, go through each item in a list. A while loop keeps running as long as a condition is True. break stops the loop immediately. continue skips the current step and goes to the next one. range(start, stop, step) generates a sequence of numbers for looping.

Example:

```
for i in range(1, 6):
    print(i) # prints 1 2 3 4 5
```

Questions — Section 2 : Conditions and Loops

6. Check if a number is positive, negative, or zero.

```
num = int(input("Enter a number: "))
if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
```

```
else:  
    print("Zero")
```

7. Print multiplication table of a number using for loop.

```
n = int(input("Enter number: "))  
for i in range(1, 11):  
    print(f"{n} x {i} = {n*i}")
```

8. Find the sum of digits of a number using while loop.

```
num = int(input("Enter number: "))  
total = 0  
while num > 0:  
    total += num % 10  
    num //= 10  
print("Sum of digits =" , total)
```

9. Print a star pattern using nested loops.

```
for i in range(1, 6):  
    print("*" * i)  
# *  
# **  
# ***
```

10. Skip even numbers and stop the loop when i reaches 15.

```
for i in range(1, 20):  
    if i == 15: break  
    if i % 2 == 0: continue  
    print(i) # prints only odd numbers before 15
```

Section 3 : Functions and Lambda

What is a Function?

A function is a reusable block of code that performs a specific task. Instead of writing the same code again and again, you define it once using the `def` keyword and call it whenever needed. A function can accept inputs (called parameters) and return an output using the `return` keyword.

Example:

```
def greet(name):  
    return "Hello, " + name  
print(greet("Jarina")) # Hello, Jarina
```

What is a Lambda Function?

A lambda is a small, one-line anonymous function — meaning it has no name. It is written using the `lambda` keyword and is useful when you need a quick function without formally defining it with `def`. Syntax: `lambda arguments : expression`

Example:

```
square = lambda x: x ** 2  
print(square(5)) # 25
```

What is Recursion?

Recursion means a function that calls itself. It is used to solve problems that can be broken down into smaller versions of the same problem. Every recursive function must have a base case — a condition where it stops calling itself, otherwise it will run forever.

Example:

```
def countdown(n):  
    if n == 0: print("Done!"); return  
    print(n)  
    countdown(n - 1)
```

Questions — Section 3 : Functions and Lambda

11. Write a function that checks if a number is even or odd.

```
def check_even_odd(n):
```

```
return "Even" if n % 2 == 0 else "Odd"
print(check_even_odd(9)) # Odd
```

12. Write a function with a default parameter to greet a user.

```
def greet(name="Guest"):
    print(f"Welcome, {name}!")
greet() # Welcome, Guest!
greet("Jarina") # Welcome, Jarina!
```

13. Use *args to accept any number of values and return their sum.

```
def total(*args):
    return sum(args)
print(total(10, 20, 30, 40)) # 100
```

14. Use lambda to sort a list of tuples by the second value.

```
students = [("Alice", 85), ("Bob", 72), ("Carol", 91)]
students.sort(key=lambda x: x[1])
print(students) # sorted by marks
```

15. Write a recursive function to calculate factorial.

```
def factorial(n):
    if n == 0: return 1
    return n * factorial(n - 1)
print(factorial(6)) # 720
```


Section 4 : Strings

What is a String?

A string is a sequence of characters stored inside quotes. Every character in a string has an index starting from 0. Strings are immutable — you cannot change a character directly, but you can create a new string. Python provides many built-in string methods: `upper()`, `lower()`, `strip()`, `replace()`, `split()`, `join()`, `find()`, `count()`, `startswith()`, `endswith()`.

Slicing Syntax: `string[start : stop : step]`

```
s = "Python"
print(s[0:3]) # Pyt
print(s[::-1]) # nohtyP (reversed)
```

Questions — Section 4 : Strings

16. Reverse a string and check if it is a palindrome.

```
s = input("Enter a string: ")
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

17. Count the number of vowels in a string.

```
text = input("Enter text: ")
vowels = "aeiouAEIOU"
count = sum(1 for c in text if c in vowels)
print("Vowel count:" , count)
```

18. Replace a word in a sentence and convert to uppercase.

```
s = "I love Java programming"
s = s.replace("Java", "Python")
print(s.upper()) # I LOVE PYTHON PROGRAMMING
```

19. Split a sentence into words and join them with a hyphen.

```
sentence = "Data Analyst Python SQL"
words = sentence.split()
result = "-".join(words)
print(result) # Data-Analyst-Python-SQL
```

20. Use f-string to display a student report card.

```
name = "Jarina"
subject = "Data Analysis"
marks = 94
print(f"Student: {name} | Subject: {subject} | Marks: {marks}/100")
```

Section 5 : Lists, Tuples, Sets and Dictionaries

What is a List?

A list is an ordered, changeable collection that can hold multiple values of any type. Lists are written with square brackets `[]`. You can add, remove, and update items. Common methods: `append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`, `len()`.

Example:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits[0]) # apple
```

What is a Tuple?

A tuple is an ordered, unchangeable (immutable) collection written with round brackets `()`. Once created, you cannot add or remove items. Tuples are used when data should not be modified — like coordinates, database records, or fixed settings.

Example:

```
student = ("Jarina", 21, "B.Com")
print(student[0]) # Jarina
```

What is a Set?

A set is an unordered collection that stores only unique values — duplicates are automatically removed. Sets are written with curly brackets `{}`. They support mathematical operations: union (`|`), intersection (`&`), difference (`-`).

Example:

```
nums = {1, 2, 2, 3, 3, 4}
print(nums) # {1, 2, 3, 4} duplicates removed
```

What is a Dictionary?

A dictionary stores data as key-value pairs — like a real dictionary where a word (key) has a meaning (value). Dictionaries are written with curly brackets `{}` using a colon `:` to separate key and value. Keys must be unique. Common methods: `keys()`, `values()`, `items()`, `get()`, `update()`, `pop()`.

Example:

```
emp = {"name": "Jarina", "salary": 35000}
```

```
print(emp["name"]) # Jarina
print(emp.get("salary")) # 35000
```

Questions — Section 5 : Lists, Tuples, Sets and Dictionaries

21. Add, remove, and sort elements in a list.

```
marks = [85, 72, 91, 60, 78]
marks.append(95)
marks.remove(60)
marks.sort()
print(marks)
```

22. Use list comprehension to get squares of even numbers from 1 to 10.

```
squares = [x**2 for x in range(1,11) if x%2==0]
print(squares) # [4, 16, 36, 64, 100]
```

23. Store a student record in a tuple and display it.

```
student = ("Jarina", 21, "B.Com", 92.5)
print(f>Name: {student[0]}, Age: {student[1]}, Course:
{student[2]}, CGPA: {student[3]}")
```

24. Use sets to find union, intersection, and difference of two groups.

```
python_students = {"Alice", "Bob", "Carol"}
sql_students = {"Bob", "Carol", "David"}
print(python_students | sql_students) # Union
print(python_students & sql_students) # Intersection
print(python_students - sql_students) # Only in Python
```

25. Create an employee dictionary and display all keys, values, and items.

```
emp = {"name": "Jarina", "dept": "Data", "salary":
35000}
print(emp.keys())
print(emp.values())
```

```
for k, v in emp.items():  
    print(f"{k}: {v}")
```

Section 6 : OOPs — Object Oriented Programming

What is OOPs?

Object Oriented Programming (OOPs) is a programming approach where we model real-world things as objects in code. For example, a Car is an object — it has properties like color and speed (called attributes) and actions like start() and stop() (called methods). OOPs has 4 main pillars: Encapsulation, Inheritance, Polymorphism, and Abstraction.

What is a Class and Object?

A class is a blueprint or template — it defines what an object will look like and what it can do. An object is an actual instance created from that class. For example: 'Employee' is a class. 'Jarina' is an object of that class. `__init__` is a special method called automatically when an object is created. `self` refers to the current object.

Example:

```
class Car():
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
    def show(self):
        print(f"{self.brand} is {self.color}")
c1 = Car("Toyota", "Red")
c1.show() # Toyota is Red
```

What is Inheritance?

Inheritance allows one class (child) to acquire the properties and methods of another class (parent). This promotes code reuse. The child class can also have its own additional methods. `super()` is used to call the parent class constructor or method.

What is Encapsulation?

Encapsulation means hiding the internal data of an object and allowing access only through methods. In Python, a private variable is created by adding double underscore `__` before the name. This protects the data from being changed accidentally from outside the class.

What is Polymorphism?

Polymorphism means the same method name works differently in different classes. For example, both Dog and Cat have a speak() method — but Dog says 'Woof' and Cat says 'Meow'. This makes code flexible and easy to extend.

Questions — Section 6 : OOPs

26. Create an Employee class with name and salary, and display details.

```
class Employee():
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display(self):
        print(f"Name: {self.name} | Salary: {self.salary}")
e1 = Employee("Jarina", 35000)
e1.display()
```

27. Demonstrate Inheritance — Manager class extends Employee.

```
class Manager(Employee):
    def __init__(self, name, salary, dept):
        super().__init__(name, salary)
        self.dept = dept
    def display(self):
        super().display()
        print("Department:" , self.dept)
m1 = Manager("Rahul", 70000, "IT")
m1.display()
```

28. Demonstrate Polymorphism — same method name in different classes.

```
class Dog():
    def speak(self): print("Woof!")
class Cat():
    def speak(self): print("Meow!")
for animal in [Dog(), Cat()]:
```

```
animal.speak()
```

29. Demonstrate Encapsulation using a private variable in BankAccount.

```
class BankAccount():  
    def __init__(self, balance):  
        self.__balance = balance # private variable  
    def deposit(self, amt):  
        self.__balance += amt  
    def get_balance(self):  
        return self.__balance  
acc = BankAccount(10000)  
acc.deposit(5000)  
print(acc.get_balance()) # 15000
```

30. Create a class with a class method and a static method.

```
class MathHelper():  
    company = "TechCorp"  
    @classmethod  
    def get_company(cls): return cls.company  
    @staticmethod  
    def add(a, b): return a + b  
print(MathHelper.get_company()) # TechCorp  
print(MathHelper.add(10, 20)) # 30
```


Section 7 : File Handling

What is File Handling?

File handling allows your Python program to read from and write to files stored on your computer. The `open()` function is used to open a file. It takes two arguments: the file name and the mode. Modes: 'r' = read, 'w' = write (creates new or overwrites), 'a' = append (adds to existing file), 'r+' = read and write both. Using `with` keyword is best practice because it automatically closes the file after use.

Example:

```
with open("notes.txt", "w") as f:
    f.write("Hello from Python")
```

Questions — Section 7 : File Handling

31. Write data to a file and read it back.

```
with open("data.txt", "w") as f:
    f.write("Name: Jarina\nCourse: Python\nScore: 95")
with open("data.txt", "r") as f:
    print(f.read())
```

32. Append a new line to an existing file without deleting old content.

```
with open("data.txt", "a") as f:
    f.write("\nNew Entry Added")
```

33. Read a file line by line and count the total number of lines.

```
with open("data.txt") as f:
    lines = f.readlines()
    print(f"Total lines: {len(lines)}")
```

Section 8 : Exception Handling

What is Exception Handling?

When a program encounters an unexpected situation at runtime, it raises an Exception (error) and stops. Exception handling allows us to catch these errors and handle them gracefully instead of crashing. try block contains the code that might cause an error. except block handles the error if it occurs. else block runs only if no error occurred. finally block always runs — whether there was an error or not. Common exceptions: ZeroDivisionError, ValueError, FileNotFoundError, IndexError, KeyError.

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Questions — Section 8 : Exception Handling

34. Handle multiple exceptions — ZeroDivisionError and ValueError.

```
try:
    n = int(input("Enter a number: "))
    result = 100 / n
    print("Result:" , result)
except ValueError:
    print("Invalid input. Please enter a number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Execution complete.")
```

35. Create a custom exception for invalid age input.

```
class InvalidAgeError(Exception):
    pass
def validate_age(age):
```

```
if age < 0 or age > 120:  
    raise InvalidAgeError("Age must be between 0 and 120.")  
print("Valid age:" , age)  
try:  
    validate_age(-3)  
except InvalidAgeError as e:  
    print(e)
```

Section 9 : Modules and Libraries

What is a Module?

A module is a Python file containing ready-made functions and classes that you can use in your program. Instead of writing everything from scratch, you import a module and use its functions. Built-in modules come with Python: math, random, datetime, os, sys. External libraries must be installed using pip: pandas, numpy, matplotlib. Syntax: import module_name OR from module_name import function_name

Example:

```
import math
print(math.sqrt(81)) # 9.0
```

Questions — Section 9 : Modules and Libraries

36. Use math module — find square root, power, floor and ceiling.

```
import math
print(math.sqrt(144)) # 12.0
print(math.pow(2, 10)) # 1024.0
print(math.floor(7.9)) # 7
print(math.ceil(7.1)) # 8
```

37. Use random module — generate a 4-digit OTP and pick a random item.

```
import random
otp = random.randint(1000, 9999)
print("Your OTP:" , otp)
items = ["Python", "SQL", "Excel", "Power BI"]
print("Today study:" , random.choice(items))
```

38. Use datetime to display today's date and calculate days until year-end.

```
from datetime import datetime
today = datetime.now()
year_end = datetime(today.year, 12, 31)
days_left = (year_end - today).days
```

```
print(f"Today: {today.strftime('%d-%m-%Y')}")  
print(f"Days until year end: {days_left}")
```

Section 10 : Advanced Python

What is a Decorator?

A decorator is a function that wraps another function and adds extra behavior to it without changing the original function's code. You apply a decorator using the @ symbol above a function definition. Common use cases: logging, timing, authentication.

What is a Generator?

A generator is a special function that yields values one at a time instead of returning all at once. It uses the yield keyword instead of return. Generators are memory-efficient — they do not store all values in memory at once. They are ideal for working with large data streams.

What is the Collections Module?

The collections module provides specialized data structures beyond the built-in ones. Counter — counts how many times each element appears. defaultdict — a dictionary that provides a default value for missing keys. namedtuple — a tuple where each value has a name.

What is a Regular Expression (Regex)?

A regular expression is a pattern used to search, match, or validate text. Python's re module provides functions like match(), search(), findall(), sub(). Common patterns: \d = digit, \w = word character, + = one or more, * = zero or more, ^ = start, \$ = end.

Questions — Section 10 : Advanced Python

39. Create a decorator that logs when a function is called and when it finishes.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Function {func.__name__} completed.")
    return result
```

```

return wrapper
@logger
def add(a, b):
    print(a + b)
    add(10, 20)

```

40. Create a generator that yields the Fibonacci sequence.

```

def fibonacci(limit):
    a, b = 0, 1
    while a < limit:
        yield a
        a, b = b, a + b
for num in fibonacci(100):
    print(num, end=" ")
# 0 1 1 2 3 5 8 13 21 34 55 89

```

41. Use Counter to find the top 3 most common words in a sentence.

```

from collections import Counter
text = "python is great python is easy python is fun"
words = text.split()
count = Counter(words)
print(count.most_common(3))
# [('python', 3), ('is', 3), ('great', 1)]

```

42. Use map() and filter() to process a list of salaries.

```

salaries = [25000, 32000, 18000, 45000, 15000, 60000]
# Give 10% raise to everyone using map()
raised = list(map(lambda x: x * 1.10, salaries))
# Filter only salaries above 30000
high = list(filter(lambda x: x > 30000, salaries))
print("After raise:" , raised)
print("High earners:" , high)

```

43. Use regex to validate an email and a 10-digit Indian phone number.

```
import re
email = "jarinachoudhary708@gmail.com"
phone = "9876543210"
email_pattern = r"^[\\w.+\\-]+@[\\w-]+\\. [a-zA-Z]{2,}\\$"
phone_pattern = r"^[6-9]\\d{9}\\$"
print("Email valid:" , bool(re.match(email_pattern,
email)))
print("Phone valid:" , bool(re.match(phone_pattern,
phone)))
```

44. Use itertools to generate all combinations and permutations.

```
from itertools import combinations, permutations
items = ["A", "B", "C"]
print("--- Combinations ---")
for c in combinations(items, 2):
    print(c)
print("--- Permutations ---")
for p in permutations(items, 2):
    print(p)
```

45. Implement Binary Search — find the position of a target in a sorted list.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            print(f"Found {target} at index {mid}")
            return
        elif arr[mid] < target: low = mid + 1
        else : high = mid - 1
    print("Target not found")
data = [5, 12, 23, 38, 45, 67, 89, 100]
```



```
binary_search(data, 45) # Found 45 at index 4  
binary_search(data, 99) # Target not found
```